

MANUAL DE BASE DE DADOS

SQL Queries e Terminal (Prisma)

PostgreSQL via Neon — ORM: Prisma 7

INDICE

1. Comandos de Terminal (Prisma CLI)
2. Tabela Negocio
3. Tabela Empleado (Funcionario)
4. Tabela Servicio (Servico)
5. Tabela Horario
6. Tabela Pausa
7. Tabela DiaBloqueado
8. Tabela Cita (Marcacao)
9. Queries Avancados
10. Estructura completa do esquema

■ 1. Comandos de Terminal (Prisma CLI)

Comandos uteis para gerir a base de dados localmente:

```
# Ver e editar dados visualmente no browser
npx prisma studio

# Sincronizar o schema com a base de dados (sem migrations)
npx prisma db push

# Criar uma migration e aplicar
npx prisma migrate dev --name nome_da_migracao

# Aplicar migrations em producao
npx prisma migrate deploy

# Regenerar o cliente Prisma apos mudar o schema
npx prisma generate

# Ver estado das migrations
npx prisma migrate status

# Reset completo (APAGA TODOS OS DADOS)
npx prisma migrate reset

# Introspect – importar schema duma BD existente
npx prisma db pull

# Formatar o schema.prisma
npx prisma format
```

■ 2. Tabela Negocio

Cada negocio e um tenant independente com a sua propria API key. Todas as outras entidades pertencem a um negocio.

```
-- SCHEMA
id Int @id @default(autoincrement())
nombre String
email String @unique
apiKey String @unique @default(cuid())
adminPassword String?
creadoEn DateTime @default(now())
```

Queries SQL:

```
-- CREATE
INSERT INTO "Negocio" (nombre, email)
VALUES ('La Patita Pet', 'email@example.com');
-- Nota: apiKey e gerado automaticamente (cuid)

-- READ – todos
SELECT id, nombre, email, "apiKey", "creadoEn" FROM "Negocio";

-- READ – por ID
SELECT * FROM "Negocio" WHERE id = 1;

-- READ – por email
SELECT * FROM "Negocio" WHERE email = 'email@example.com';

-- READ – por apiKey
SELECT * FROM "Negocio" WHERE "apiKey" = 'cmpymk127000004ju..';

-- UPDATE – nome
UPDATE "Negocio" SET nombre = 'Novo Nome' WHERE id = 1;

-- UPDATE – email
UPDATE "Negocio" SET email = 'novo@email.com' WHERE id = 1;

-- DELETE (cuidado: elimina tudo em cascata)
DELETE FROM "Negocio" WHERE id = 1;
```

■ 3. Tabela Empleado (Funcionario)

```
-- SCHEMA
id Int @id @default(autoincrement())
negocioId Int (FK -> Negocio)
nombre String
email String?
telefono String?
creadoEn DateTime @default(now())
```

```
-- CREATE
INSERT INTO "Empleado" ("negocioId", nombre, email, telefono)
VALUES (1, 'Maria Garcia', 'maria@email.com', '+351 912 345 678');

-- READ – todos do negocio
SELECT * FROM "Empleado" WHERE "negocioId" = 1 ORDER BY nombre;

-- READ – por ID
SELECT * FROM "Empleado" WHERE id = 5;

-- UPDATE
UPDATE "Empleado"
SET nombre = 'Maria Silva', email = 'silva@email.com'
WHERE id = 5;

-- DELETE
DELETE FROM "Empleado" WHERE id = 5;

-- COUNT por negocio
SELECT COUNT(*) FROM "Empleado" WHERE "negocioId" = 1;
```

➤ 4. Tabela Servicio (Servico)

```
-- SCHEMA
id Int @id @default(autoincrement())
empleadoId Int (FK -> Empleado)
nombre String
duracion Int (minutos)
precio Float?
```

```
-- CREATE
INSERT INTO "Servicio" ("empleadoId", nombre, duracion, precio)
VALUES (5, 'Banho & Brilho', 60, 25.00);

-- READ – todos do funcionario
SELECT * FROM "Servicio" WHERE "empleadoId" = 5;

-- READ – por ID
SELECT * FROM "Servicio" WHERE id = 10;

-- READ – servicios com preco
SELECT * FROM "Servicio" WHERE "empleadoId" = 5 AND precio IS NOT NULL;

-- UPDATE
UPDATE "Servicio"
SET nombre = 'Banho Completo', duracion = 90, precio = 30.00
WHERE id = 10;

-- DELETE
DELETE FROM "Servicio" WHERE id = 10;

-- Verificar se tem marcacoes (antes de eliminar)
SELECT COUNT(*) FROM "Cita" WHERE "servicioId" = 10;
```

■ 5. Tabela Horario

```
-- SCHEMA
id Int @id @default(autoincrement())
empleadoId Int (FK -> Empleado)
diaSemana String ('lunes','martes','miercoles','jueves',
'viernes','sabado','domingo')
horaInicio String ('HH:MM')
horaFin String ('HH:MM')

-- CREATE
INSERT INTO "Horario" ("empleadoId", "diaSemana", "horaInicio", "horaFin")
VALUES (5, 'lunes', '09:00', '18:00');

-- READ – todos do funcionario
SELECT * FROM "Horario" WHERE "empleadoId" = 5;

-- READ – horario de um dia especifico
SELECT * FROM "Horario"
WHERE "empleadoId" = 5 AND "diaSemana" = 'lunes';

-- READ – todos os funcionarios com horario a segunda
SELECT e.nombre, h."horaInicio", h."horaFin"
FROM "Horario" h JOIN "Empleado" e ON h."empleadoId" = e.id
WHERE h."diaSemana" = 'lunes' AND e."negocioId" = 1;

-- UPDATE
UPDATE "Horario"
SET "horaInicio" = '08:00', "horaFin" = '17:00'
WHERE id = 15;

-- DELETE – por ID
DELETE FROM "Horario" WHERE id = 15;

-- DELETE – dia especifico de um funcionario
DELETE FROM "Horario"
WHERE "empleadoId" = 5 AND "diaSemana" = 'lunes';
```

■ 6. Tabela Pausa

```
-- SCHEMA
id Int @id @default(autoincrement())
empleadoId Int (FK -> Empleado)
diaSemana String (mesmo formato que Horario)
horaInicio String ('HH:MM')
horaFin String ('HH:MM')
```

```
-- CREATE
INSERT INTO "Pausa" ("empleadoId", "diaSemana", "horaInicio", "horaFin")
VALUES (5, 'lunes', '13:00', '14:00');

-- READ – todas as pausas do funcionario
SELECT * FROM "Pausa" WHERE "empleadoId" = 5;

-- READ – pausas de um dia especifico
SELECT * FROM "Pausa"
WHERE "empleadoId" = 5 AND "diaSemana" = 'lunes';

-- DELETE
DELETE FROM "Pausa" WHERE id = 20;

-- DELETE – todas as pausas de um dia
DELETE FROM "Pausa"
WHERE "empleadoId" = 5 AND "diaSemana" = 'sabado';
```

■ 7. Tabela DiaBloqueado

```
-- SCHEMA
id Int @id @default(autoincrement())
empleadoId Int (FK -> Empleado)
fecha DateTime
motivo String?
```

```
-- CREATE
INSERT INTO "DiaBloqueado" ("empleadoId", fecha, motivo)
VALUES (5, '2025-12-25', 'Natal');
```

```
-- READ – todos (futuros)
SELECT * FROM "DiaBloqueado"
WHERE "empleadoId" = 5 AND fecha > NOW()
ORDER BY fecha ASC;
```

```
-- READ – verificar se uma data esta bloqueada
SELECT * FROM "DiaBloqueado"
WHERE "empleadoId" = 5
AND DATE(fecha) = '2025-12-25';
```

```
-- DELETE – desbloquear
DELETE FROM "DiaBloqueado" WHERE id = 25;
```

```
-- DELETE – limpar dias passados
DELETE FROM "DiaBloqueado" WHERE fecha < NOW();
```

```
-- DELETE – bloquear um intervalo e depois apagar
DELETE FROM "DiaBloqueado"
WHERE "empleadoId" = 5
AND fecha BETWEEN '2025-08-01' AND '2025-08-15';
```

■ 8. Tabela Cita (Marcacao)

```
-- SCHEMA
id Int @id @default(autoincrement())
negocioId Int (FK -> Negocio)
empleadoId Int (FK -> Empleado)
servicioId Int (FK -> Servicio)
nombreCliente String
emailCliente String
fecha DateTime ('YYYY-MM-DDTHH:MM')
estado String @default('pendiente')
-- valores: 'pendiente' | 'confirmada' | 'cancelada'
creadoEn DateTime @default(now())
```

```
-- CREATE
INSERT INTO "Cita"
("negocioId", "empleadoId", "servicioId", "nombreCliente", "emailCliente", fecha)
VALUES (1, 5, 10, 'Ana Costa', 'ana@email.com', '2025-06-20T10:00');

-- READ – todas com joins
SELECT c.id, c."nombreCliente", c."emailCliente", c.fecha, c.estado,
s.nombre AS servicio, s.duracion, e.nombre AS especialista
FROM "Cita" c
JOIN "Servicio" s ON c."servicioId" = s.id
JOIN "Empleado" e ON c."empleadoId" = e.id
WHERE c."negocioId" = 1
ORDER BY c.fecha DESC;

-- READ – por data especifica
SELECT * FROM "Cita"
WHERE "negocioId" = 1 AND DATE(fecha) = '2025-06-20';

-- READ – pendientes
SELECT * FROM "Cita"
WHERE "negocioId" = 1 AND estado = 'pendiente'
ORDER BY fecha ASC;

-- READ – de hoje
SELECT * FROM "Cita"
WHERE "negocioId" = 1 AND DATE(fecha) = CURRENT_DATE;

-- READ – de um funcionario especifico
SELECT * FROM "Cita"
WHERE "empleadoId" = 5 AND "negocioId" = 1
ORDER BY fecha DESC;

-- UPDATE – mudar estado
UPDATE "Cita" SET estado = 'confirmada' WHERE id = 30;
UPDATE "Cita" SET estado = 'cancelada' WHERE id = 30;

-- DELETE
DELETE FROM "Cita" WHERE id = 30;

-- DELETE – canceladas mais antigas de 30 dias
DELETE FROM "Cita"
WHERE estado = 'cancelada'
AND fecha < NOW() - INTERVAL '30 days';
```

■ 9. Queries Avancados

```
-- ESTATISTICAS: contagem por estado
SELECT estado, COUNT(*) AS total
FROM "Cita" WHERE "negocioId" = 1
GROUP BY estado;

-- ESTATISTICAS: marcacoes por dia (ultimos 30 dias)
SELECT DATE(fecha) AS dia, COUNT(*) AS total
FROM "Cita" WHERE "negocioId" = 1
AND fecha >= NOW() - INTERVAL '30 days'
GROUP BY DATE(fecha) ORDER BY dia;

-- ESTATISTICAS: servicios mais reservados
SELECT s.nombre, COUNT(*) AS total
FROM "Cita" c JOIN "Servicio" s ON c."servicioId" = s.id
WHERE c."negocioId" = 1
GROUP BY s.nombre ORDER BY total DESC;

-- DISPONIBILIDADE: horas ocupadas num dia/funcionario
SELECT fecha FROM "Cita"
WHERE "empleadoId" = 5
AND DATE(fecha) = '2025-06-20'
AND estado != 'cancelada';

-- MARCACOES desta semana de um funcionario
SELECT c.*, s.nombre AS servicio
FROM "Cita" c JOIN "Servicio" s ON c."servicioId" = s.id
WHERE c."empleadoId" = 5
AND c.fecha >= date_trunc('week', NOW())
AND c.fecha < date_trunc('week', NOW()) + INTERVAL '7 days';

-- TODOS OS NEGOCIOS com contagem de citas
SELECT n.nombre, n.email, COUNT(c.id) AS total_citas
FROM "Negocio" n
LEFT JOIN "Cita" c ON c."negocioId" = n.id
GROUP BY n.id, n.nombre, n.email;

-- VERIFICAR CONFLITO de horario (sobreposicao)
SELECT * FROM "Cita"
WHERE "empleadoId" = 5
AND DATE(fecha) = '2025-06-20'
AND fecha BETWEEN '2025-06-20T10:00' AND '2025-06-20T11:00'
AND estado != 'cancelada';
```

■ 10. Estructura completa do esquema

```
model Negocio {
  id Int @id @default(autoincrement())
  nombre String
  email String @unique
  apiKey String @unique @default(cuid())
  adminPassword String?
  creadoEn DateTime @default(now())
  empleados Empleado[]
  citas Cita[]
}

model Empleado {
  id Int @id @default(autoincrement())
  negocioId Int
  nombre String
  email String?
  telefono String?
  creadoEn DateTime @default(now())
  negocio Negocio @relation(fields:[negocioId], references:[id])
  servicios Servicio[]
  horarios Horario[]
  pausas Pausa[]
  diasBloqueados DiaBloqueado[]
  citas Cita[]
}

model Servicio {
  id Int @id @default(autoincrement())
  empleadoId Int
  nombre String
  duracion Int
  precio Float?
  empleado Empleado @relation(fields:[empleadoId], references:[id])
  citas Cita[]
}
```

```
model Horario {
  id Int @id @default(autoincrement())
  empleadoId Int
  diaSemana String
  horaInicio String
  horaFin String
  empleado Empleado @relation(fields:[empleadoId], references:[id])
}

model Pausa {
  id Int @id @default(autoincrement())
  empleadoId Int
  diaSemana String
  horaInicio String
  horaFin String
  empleado Empleado @relation(fields:[empleadoId], references:[id])
}

model DiaBloqueado {
  id Int @id @default(autoincrement())
  empleadoId Int
  fecha DateTime
  motivo String?
  empleado Empleado @relation(fields:[empleadoId], references:[id])
}

model Cita {
  id Int @id @default(autoincrement())
  negocioId Int
  empleadoId Int
  servicioId Int
  nombreCliente String
  emailCliente String
  fecha DateTime
  estado String @default('pendiente')
  creadoEn DateTime @default(now())
  negocio Negocio @relation(fields:[negocioId], references:[id])
  empleado Empleado @relation(fields:[empleadoId], references:[id])
  servicio Servicio @relation(fields:[servicioId], references:[id])
}
```